

Guide complet de développement

Site de gestion des demandes d'accréditation presse — Basketball & Handball

Ce document reprend intégralement les 10 prompts proposés pour construire le site étape par étape. L'objectif est de pouvoir les copier-coller successivement dans un agent de développement comme Cursor, Codex, Claude Code ou équivalent.

Architecture recommandée

Stack recommandée : Next.js + TypeScript + Tailwind CSS + MongoDB/Mongoose, avec authentification admin sécurisée.

[illegible]

Étape 1 — Faire construire l'architecture du projet

Je souhaite développer une application web complète de gestion des demandes d'accréditation presse pour des matchs de basketball et de handball.

L'application doit être professionnelle, responsive, moderne et simple à utiliser.

Stack technique souhaitée

Utilise :

- Next.js avec TypeScript
- App Router
- Tailwind CSS
- MongoDB
- Mongoose
- API Routes / Route Handlers Next.js
- Authentification sécurisée pour l'espace administrateur
- Architecture propre et maintenable

Si une autre technologie est nécessaire, explique pourquoi avant de l'utiliser.

Fonctionnement général

Le site possède deux parties :

1. Un espace public destiné aux journalistes, photographes et vidéastes.
2. Un espace privé d'administration permettant de gérer les demandes.

Navigation publique

Le site doit avoir deux catégories principales :

- Basketball
- Handball

Chaque catégorie affiche les matchs correspondants.

Exemple :

Basketball

- Équipe A vs Équipe B
- Date
- Heure
- Lieu
- Statut des inscriptions

- Nombre de places / demandes si cette information doit être publique

Handball

- Équipe A vs Équipe B
- Date
- Heure
- Lieu
- Statut des inscriptions

Lorsqu'un utilisateur clique sur un match, il arrive sur une page détaillée du match avec le formulaire de demande d'accréditation.

Types de demandes

Il existe trois profils :

- Journaliste
- Photographe
- Vidéaste

Pour un journaliste :

- Nom
- Prénom
- Adresse e-mail
- Carte de presse / scan de carte de presse, facultatif
- Besoin d'un chasuble : oui/non

Pour un photographe :

- Nom
- Prénom
- Adresse e-mail
- Lien vers le portfolio, facultatif mais fortement recommandé

Pour un vidéaste :

- Nom
- Prénom
- Adresse e-mail
- Lien vers le portfolio, facultatif mais fortement recommandé

Administration

L'administrateur doit pouvoir :

- se connecter ;
- voir les matchs ;
- créer un match ;
- modifier un match ;
- supprimer / désactiver un match ;
- voir les demandes d'accréditation ;
- filtrer les demandes par type ;
- accepter une demande ;
- refuser une demande ;
- voir le nombre total de demandes par match ;
- voir le nombre de demandes acceptées ;
- voir le nombre de demandes refusées ;
- voir les demandes en attente.

Les statuts d'une demande doivent être :

- PENDING
- ACCEPTED
- REJECTED

Règle importante

Les inscriptions doivent automatiquement être fermées 24 heures avant le début du match.

Exemple :

Match le samedi à 20h00.

Les inscriptions sont possibles jusqu'au vendredi à 20h00.

Après cette date, le formulaire doit être désactivé et l'utilisateur doit voir un message indiquant que les inscriptions sont closes.

Cette règle doit être vérifiée côté serveur et pas uniquement côté frontend.

Base de données

Toutes les données doivent être stockées dans MongoDB.

Prévoir au minimum les collections :

- matches
- accreditationRequests
- admins

Créer une architecture permettant d'ajouter facilement d'autres collections par la suite.

Important

Avant de coder, commence par :

1. proposer l'architecture des dossiers ;
2. proposer les modèles MongoDB ;
3. proposer les routes API ;
4. proposer les pages frontend ;
5. expliquer les relations entre les différentes données ;
6. proposer les règles de sécurité.

Ne commence pas immédiatement à générer tout le code.

Présente d'abord l'architecture complète et attends ma validation.

Étape 2 — Faire créer la base MongoDB

Maintenant, implémente la partie MongoDB de l'application.

Utilise MongoDB avec Mongoose.

Je veux au minimum les modèles suivants.

Match

Un match doit contenir :

- sport : BASKETBALL | HANDBALL
- homeTeam
- awayTeam
- date
- startTime ou une date complète permettant de calculer précisément l'heure du match
- venue
- description éventuelle
- status : UPCOMING | FINISHED | CANCELLED
- accreditationDeadline
- createdAt
- updatedAt

L'accreditationDeadline doit être calculée automatiquement à partir de la date et de l'heure du match :

matchDateTime - 24 heures.

Prévoir également la possibilité de définir manuellement cette date si nécessaire, mais la valeur par défaut doit être automatiquement calculée.

AccreditationRequest

Une demande doit contenir :

- matchId
- sport
- profileType : JOURNALIST | PHOTOGRAPHER | VIDEOGRAPHER
- firstName
- lastName
- email
- pressCardUrl éventuellement
- needsBib éventuellement
- portfolioUrl éventuellement

- status : PENDING | ACCEPTED | REJECTED
- rejectionReason éventuellement
- createdAt
- updatedAt
- processedAt éventuellement

Admin

Prévoir :

- email
- passwordHash
- role
- createdAt
- updatedAt

Ne jamais stocker le mot de passe en clair.

Contraintes

Ajouter les validations nécessaires.

Par exemple :

- email valide ;
- portfolioUrl valide si renseigné ;
- pressCardUrl valide si renseigné ;
- firstName obligatoire ;
- lastName obligatoire ;
- email obligatoire ;
- profileType obligatoire ;
- matchId obligatoire.

Créer également les index MongoDB nécessaires pour les recherches fréquentes :

- matchId
- sport
- status
- email
- date du match

Prévoir une protection contre les doublons évidents.

Par exemple, empêcher qu'une même adresse e-mail puisse envoyer plusieurs demandes identiques pour le même match, sauf si nous décidons explicitement d'autoriser plusieurs profils.

Implémente également la connexion MongoDB de manière propre avec réutilisation de la connexion en environnement de développement.

Crée les fichiers nécessaires et explique brièvement chaque fichier créé.

Étape 3 — Créer l'API

Maintenant, implémente l'API complète de l'application.

L'API doit permettre de gérer les matchs et les demandes d'accréditation.

API publique

Créer une route permettant de récupérer les matchs à venir.

Exemple :

GET /api/matches

Possibilité de filtrer :

- sport
- date
- statut

Créer également :

GET /api/matches/[id]

pour récupérer les informations détaillées d'un match.

Création d'une demande

Créer :

POST /api/accreditations

La route doit :

1. vérifier que le match existe ;
2. vérifier que le match est bien ouvert aux accréditations ;
3. vérifier que la date limite n'est pas dépassée ;
4. valider les données reçues ;
5. vérifier les éventuels doublons ;
6. créer la demande MongoDB ;
7. retourner une réponse claire au frontend.

IMPORTANT :

La règle des 24 heures doit impérativement être vérifiée côté serveur.

Il ne faut jamais faire confiance uniquement au frontend.

API admin

Créer les routes sécurisées permettant :

GET /api/admin/matches

POST /api/admin/matches

PUT /api/admin/matches/[id]

DELETE /api/admin/matches/[id]

GET /api/admin/accreditations

PUT /api/admin/accreditations/[id]

Les routes admin doivent nécessiter une authentification.

Filtres administrateur

L'administrateur doit pouvoir filtrer les demandes :

- par match ;
- par sport ;
- par type de profil ;
- par statut ;
- par date.

Statistiques

Créer également une API permettant d'obtenir les statistiques d'un match :

- total des demandes ;
- demandes en attente ;
- demandes acceptées ;
- demandes refusées ;
- journalistes ;
- photographes ;
- vidéastes.

Toutes les entrées doivent être validées côté serveur.

Ne jamais faire confiance aux données envoyées par le frontend.

Retourner des erreurs HTTP cohérentes et des messages exploitables par l'interface.

Étape 4 — Créer le site public

Maintenant, crée toute l'interface publique.

L'interface doit être moderne, professionnelle et responsive, avec une expérience particulièrement simple sur mobile.

Page d'accueil

Créer une page d'accueil présentant :

- le logo / nom de l'organisation ;
- une courte présentation ;
- deux grandes entrées :

■ Basketball

■ Handball

Chaque bouton redirige vers la liste des matchs correspondants.

Page Basketball

Créer :

/basketball

Cette page affiche tous les matchs de basketball à venir.

Chaque match doit être présenté sous forme de carte avec :

- équipe domicile ;
- équipe extérieure ;
- date ;
- heure ;
- lieu ;
- statut des inscriptions ;
- bouton "Demander une accréditation".

Page Handball

Créer :

/handball

Même fonctionnement que Basketball.

Page Match

Créer :

/basketball/[id]

et

/handball/[id]

La page doit afficher :

- nom des équipes ;
- date ;
- heure ;
- lieu ;
- informations importantes ;
- état des inscriptions.

Si les inscriptions sont ouvertes :

afficher un bouton permettant d'accéder au formulaire.

Si les inscriptions sont fermées :

afficher clairement :

"Les demandes d'accréditation pour ce match sont closes."

Formulaire

Créer un formulaire moderne avec une première sélection :

"Quel est votre profil ?"

Choix :

- Journaliste
- Photographe
- Vidéaste

Si Journaliste :

Afficher :

- Prénom
- Nom
- Email
- Carte de presse
- Besoin d'un chasuble : Oui / Non

Si Photographe ou Vidéaste :

Afficher :

- Prénom
- Nom
- Email
- Portfolio

Le portfolio doit être indiqué comme facultatif mais recommandé.

Validation

Ajouter une validation frontend claire.

Afficher les erreurs directement sous les champs.

Afficher un état de chargement lors de l'envoi.

Après une demande réussie, afficher une page de confirmation expliquant que la demande est en attente de validation.

Ne jamais considérer la demande comme acceptée automatiquement.

L'acceptation doit être effectuée par l'administrateur.

Le frontend doit utiliser l'API précédemment créée et ne doit jamais accéder directement à MongoDB.

Étape 5 — Ajouter l'upload de carte de presse

Maintenant, ajoute la gestion des fichiers pour les cartes de presse.

Les journalistes peuvent uploader un scan ou une photo de leur carte de presse.

Ne stocke pas le fichier directement dans MongoDB.

MongoDB doit uniquement conserver une référence vers le fichier, par exemple :

`pressCardUrl`

Le système doit :

1. accepter les formats JPG, JPEG, PNG et PDF ;
2. limiter la taille des fichiers ;
3. vérifier le type MIME ;
4. générer un nom de fichier sécurisé ;
5. stocker le fichier dans un stockage externe ;
6. enregistrer l'URL ou l'identifiant du fichier dans MongoDB.

Les fichiers contenant des données personnelles ne doivent pas être publiquement accessibles sans contrôle.

Dans l'espace admin, l'administrateur doit pouvoir cliquer sur "Voir la carte de presse" pour consulter le document.

Prévoir également une gestion propre des erreurs d'upload.

Si aucun système de stockage n'est encore configuré, créer une abstraction `StorageService` afin que le fournisseur puisse être remplacé facilement plus tard.

Étape 6 — Construire le dashboard admin

Maintenant, construis l'espace d'administration complet.

L'administration doit être accessible uniquement après authentification.

Login

Créer :

/admin/login

avec :

- email ;
- mot de passe ;
- bouton de connexion.

Ne jamais stocker les mots de passe en clair.

Mettre en place une authentification sécurisée avec session sécurisée / cookie HTTP-only.

Dashboard

Créer :

/admin

Afficher des statistiques générales :

- Nombre de matchs à venir
- Nombre de demandes en attente
- Nombre de demandes acceptées
- Nombre de demandes refusées

Navigation

Le dashboard doit comporter :

- Dashboard
- Basketball
- Handball
- Demandes
- Déconnexion

Basketball

Créer :

/admin/basketball

Afficher la liste des matchs de basketball.

Pour chaque match :

- équipes ;
- date ;
- heure ;
- lieu ;
- nombre total de demandes ;
- demandes en attente ;
- demandes acceptées ;
- demandes refusées ;
- statut des inscriptions.

Ajouter :

- créer un match ;
- modifier ;
- supprimer / désactiver ;
- consulter les demandes.

Handball

Créer :

/admin/handball

Même fonctionnement.

Page des demandes

Créer une interface permettant de visualiser les demandes d'un match.

Afficher un tableau avec :

- nom ;
- prénom ;
- email ;
- profil ;
- portfolio ;
- carte de presse ;
- chasuble ;
- date de demande ;

- statut ;
- actions.

Les profils doivent être facilement identifiables.

Ajouter des filtres :

- Tous
- En attente
- Acceptés
- Refusés
- Journalistes
- Photographes
- Vidéastes

Actions

Pour chaque demande :

"Accepter"

"Refuser"

Lors d'un refus, demander éventuellement une confirmation avant l'action.

Lors d'une acceptation ou d'un refus, mettre immédiatement à jour l'interface.

Prévoir également une confirmation visuelle de l'action.

UX

Le dashboard doit être professionnel et très lisible.

Les statistiques importantes doivent être visibles immédiatement.

Les demandes en attente doivent être particulièrement faciles à identifier.

L'interface doit fonctionner correctement sur ordinateur et tablette.

Étape 7 — Ajouter les emails automatiques

Maintenant, ajoute le système d'emails automatiques.

Utilise un service d'envoi d'emails transactionnels configurable.

Créer un EmailService indépendant du reste de l'application afin que le fournisseur d'emails puisse être changé facilement.

Acceptation

Lorsqu'un administrateur accepte une demande :

1. mettre à jour le statut MongoDB :

ACCEPTED

2. enregistrer processedAt ;

3. envoyer automatiquement un email à l'adresse du demandeur.

L'email doit contenir :

- Bonjour + prénom ;
- confirmation que la demande d'accréditation est acceptée ;
- sport ;
- match ;
- équipes ;
- date ;
- heure ;
- lieu ;
- éventuelles informations pratiques.

Créer un template email professionnel.

Refus

Lorsqu'un administrateur refuse une demande :

1. mettre à jour :

REJECTED

2. enregistrer processedAt ;

3. envoyer un email.

Le message doit expliquer poliment que la demande n'a pas pu être retenue en raison du nombre important de demandes reçues.

Important

L'envoi de l'email ne doit pas empêcher la mise à jour de la demande en base.

Prévoir une gestion des erreurs d'envoi.

Ajouter éventuellement un champ :

emailStatus

avec :

- PENDING
- SENT
- FAILED

et éventuellement :

emailSentAt

emailError

Cela permettra de savoir si l'email a bien été envoyé.

Ne jamais envoyer deux emails accidentellement lorsqu'un administrateur clique plusieurs fois sur le bouton.

Prévoir un mécanisme d'idempotence.

Étape 8 — Automatiser la fermeture des inscriptions

Implémente définitivement la logique de fermeture des inscriptions.

Pour chaque match :

`accreditationDeadline = matchDateTime - 24 heures`

Frontend

Si :

`now < accreditationDeadline`

alors afficher :

"Les demandes d'accréditation sont ouvertes."

Sinon :

"Les demandes d'accréditation sont closes."

Le formulaire ne doit plus être affiché après la deadline.

Backend

La route POST `/api/accreditations` doit également vérifier :

`now < accreditationDeadline`

Si la deadline est dépassée :

refuser la requête avec une erreur appropriée.

Cette vérification doit être faite côté serveur même si le frontend masque le formulaire.

Fuseaux horaires

Utiliser une gestion cohérente des dates et heures.

Stocker les dates en UTC dans MongoDB.

Afficher les heures dans le fuseau horaire configuré pour l'organisation.

Éviter absolument les calculs de dates avec de simples chaînes de caractères.

Ajouter des tests pour :

- match dans plus de 24 heures ;
- match exactement à 24 heures ;
- match à moins de 24 heures ;

- match déjà commencé ;
- match terminé.

Étape 9 — Sécuriser l'application

Effectue maintenant un audit de sécurité complet de l'application.

Vérifie notamment :

Authentification

- mots de passe hashés ;
- sessions sécurisées ;
- cookies HTTP-only ;
- protection CSRF si nécessaire ;
- expiration des sessions ;
- logout correctement implémenté.

API

Toutes les routes admin doivent être protégées.

Un utilisateur public ne doit jamais pouvoir :

- récupérer toutes les demandes ;
- modifier une demande ;
- accepter une demande ;
- refuser une demande ;
- créer/modifier/supprimer un match.

Données personnelles

Les demandes contiennent des données personnelles.

Ne jamais exposer publiquement :

- email ;
- carte de presse ;
- informations privées ;
- demandes d'autres utilisateurs.

Upload

Sécuriser les uploads :

- type MIME ;
- extension ;
- taille maximale ;
- nom de fichier ;

- accès aux fichiers.

MongoDB

Utiliser uniquement les variables d'environnement pour les credentials.

Ne jamais écrire les secrets directement dans le code.

Validation

Toutes les données reçues par l'API doivent être validées.

Utiliser une librairie de validation adaptée comme Zod.

Rate limiting

Ajouter une protection contre les abus sur :

POST /api/accreditations

et les routes de login.

Logs

Ne jamais logger :

- mot de passe ;
- token ;
- données sensibles ;
- contenu des cartes de presse.

Enfin, vérifie qu'il n'existe aucune route permettant de contourner les permissions administrateur.

Étape 10 — Tests

Maintenant, réalise une phase complète de tests de l'application.

Créer des tests pour les cas suivants.

Matches

- création d'un match basketball ;
- création d'un match handball ;
- modification ;
- suppression ;
- récupération ;
- filtrage par sport.

Accréditations

Tester :

- journaliste ;
- photographe ;
- vidéaste ;
- données valides ;
- données invalides ;
- email invalide ;
- portfolio invalide ;
- demande sans portfolio ;
- journaliste sans carte de presse ;
- demande avec chasuble.

Deadline

Tester :

- demande à plus de 24h ;
- demande exactement à 24h ;
- demande à moins de 24h ;
- demande après le début du match.

Admin

Tester :

- utilisateur non connecté ;

- utilisateur connecté ;
- acceptation ;
- refus ;
- double clic sur acceptation ;
- double clic sur refus ;
- accès interdit aux routes admin.

Emails

Tester :

- email d'acceptation ;
- email de refus ;
- échec d'envoi ;
- absence de double email.

Sécurité

Tester :

- accès direct aux API admin sans authentification ;
- modification d'une demande par un utilisateur public ;
- injection ;
- données invalides ;
- upload de fichiers interdits ;
- fichiers trop volumineux ;
- rate limiting.

Corriger tous les problèmes trouvés.

À la fin, donne-moi un rapport indiquant :

- tests réussis ;
- tests échoués ;
- problèmes corrigés ;
- problèmes restant à corriger ;
- recommandations avant mise en production.

Structure finale du site

```
/
■■■ accueil
■
■■■ basketball
■ ■■■ liste des matchs
■ ■■■ [matchId]
■ ■■■ formulaire d'accréditation
■
■■■ handball
■ ■■■ liste des matchs
■ ■■■ [matchId]
■ ■■■ formulaire d'accréditation
■
■■■ admin
■■■ login
■
■■■ dashboard
■
■■■ basketball
■ ■■■ matchs
■ ■■■ créer un match
■ ■■■ demandes
■
■■■ handball
■ ■■■ matchs
■ ■■■ créer un match
■ ■■■ demandes
■
■■■ demandes
```

Structure MongoDB

```
MongoDB
■
■■■ admins
■
■■■ matches
■ ■■■ Basketball
■ ■■■ Handball
■
■■■ accreditationRequests
■■■ Journalist
■■■ Photographer
■■■ Videographer
```

Recommandation importante

Je te conseille de ne pas créer trois formulaires complètement indépendants. Fais plutôt un seul formulaire dynamique : « Quel est votre profil ? » → Journaliste / Photographe / Vidéaste. Puis les champs changent en fonction du choix. Cela simplifie énormément le développement et la gestion des données.

Autre point important : les matchs doivent être créés depuis l'administration, pas codés en dur dans le site. Ainsi, l'admin pourra ajouter un nouveau match de basket ou de handball sans modifier le code.

Si tu utilises Cursor/Codex/Claude Code, donne les prompts dans cet ordre, un par un, en demandant à l'agent de ne passer à l'étape suivante qu'après avoir terminé et vérifié la précédente. C'est beaucoup plus fiable que le prompt « construis-moi tout le site ».